

Incrementally Updating High-Utility Itemsets with Transaction Insertion

Jerry Chun-Wei Lin^{1,2}, Wensheng Gan¹, Tzung-Pei Hong^{3,4}, and Jeng-Shyang Pan^{1,2}

¹ Innovative Information Industry Research Center (IIIRC)

² Shenzhen Key Laboratory of Internet Information Collaboration
School of Computer Science and Technology

Harbin Institute of Technology Shenzhen Graduate School
HIT Campus Shenzhen University Town, Xili, Shenzhen, P.R. China

³ Department of Computer Science and Information Engineering
National University of Kaohsiung, Kaohsiung, Taiwan, R.O.C.

⁴ Department of Computer Science and Engineering
National Sun Yat-sen University, Kaohsiung, Taiwan, R.O.C.
jerrylin@ieee.org, {wsgan001, jengshyangpan}@gmail.com,
tphong@nuk.edu.tw

Abstract. High-utility itemsets mining (HUIM) is designed to solve the limitations of association-rule mining by considering both the quantity and profit measures. Most algorithms of HUIM are designed to handle the static database. Fewer research handles the dynamic HUIM with transaction insertion, thus requiring the computations of database rescan and combination explosion of pattern-growth mechanism. In this paper, an efficient incremental algorithm with transaction insertion is designed to reduce computations without candidate generation based on the utility-list structures. The enumeration tree and the relationships between 2-itemsets are also adopted in the proposed algorithm to speed up computations. Several experiments are conducted to show the performance of the proposed algorithm in terms of runtime, and memory consumption.

Keywords: utility mining, transaction insertion, utility-list, enumeration tree, dynamic databases.

1 Introduction

Association-rule mining (ARM) [2, 3, 4, 8] is a fundamental task for revealing the relationships among items. Many algorithms have been respectively proposed to efficiently mine the association rules based on whether the level-wise or pattern-growth mechanism [3, 8]. Both the level-wise or pattern-growth approaches can only handle the static database in batch mode. When transactions are changed in the database, new information may arise and old ones may become invalid. The updated database is required to be processed to mine the updated information in batch mode, which is not suitable in practical applications. To solve the limitations of mining algorithms in batch mode [11, 17], Cheung et al. proposed the Fast-UPdated (FUP)

algorithm [9] to maintain and update the discovered information with transaction insertion. When the itemsets are small in the original database (support ratio is lower than minimum support threshold) but large (support ratio is larger than or equal to the minimum support threshold) in the inserted database, the original database is still required to be rescanned to find the actual occurrence frequencies of the small itemsets in the original database.

For ARM, the implicit factors such as profit or quantity are not considered. High-utility itemsets mining (HUIM) [7, 20] was thus proposed to partially solve the limitations of ARM. It may be thought of as an extension of frequent-itemset mining by considering the sold quantities and profits of the items. Several algorithms have been proposed to mine HUIs in a static database [12, 16, 18, 19]. Some HUIM algorithms have been proposed with transaction insertion [6, 13, 14]. The original database is still, however, required to be rescanned for maintaining and updating the HUIs in some cases. The problem of combination explosion based on level-wise approach is also a critical issue to be solved.

In this paper, a memory-based incremental approach is proposed with transaction insertion to efficiently discover HUIs. The proposed algorithm inherits the HUI-Miner algorithm [15] to build the utility-list structures for mining HUIs with transaction insertion. An Estimated Utility Co-Occurrence Structure (EUCS) [10] is also applied in the proposed algorithm to speed up the performance of the proposed approach. Based on the designed algorithm, it outperforms the two-phase algorithm [16] and the state-of-the-art FHM algorithm [10] in batch mode and other previous algorithms for incremental mining [13, 14].

2 Related Work

Utility mining [7, 20] is concerned as an extension of the frequent itemsets by considering both the quantities and profits of items to discover the valuable itemsets than the frequent ones. An itemset is concerned as a HUI if its utility is larger than or equal to the minimum utility count. Chan et al. first proposed the top-k objective-directed data mining to mine the top-k closed utility patterns based on business objective [7]. Yao et al. proposed the utility model to firstly consider both quantities and profits of the items to mine the HUIs [20]. Liu et al. proposed the two-phase model [16] to mine HUIs based on the developed transaction-weighted downward closure (TWDC) property. Many algorithms been proposed to mine HUIs based on two-phase model. Lin et al. designed a high utility pattern tree (HUP-tree) algorithm [12] to compress the original database into a tree structure. Tseng et al. then proposed the UP-tree structure with UP-growth and UP-growth+ mining algorithms to efficiently mine HUIs [18]. Liu et al. then proposed a HUI-Miner algorithm [15] to compress the database into the utility-list structures. Each entry in the utility-list structure stores transaction IDs (TIDs), the utility of itemset X in the transaction ($I_{utility}$), and the rest utilities of itemsets except X in the transaction ($R_{utility}$). Based on the HUI-Miner algorithm and the designed pruning strategy of the enumeration tree, the HUIs can be easily discovered. Fournier-Viger et al. then modified the

HUI-Miner algorithm and designed an Estimated Utility Co-Occurrence Structure (EUCS) to keep the relationships between 2-itemsets, thus speeding up the computations compared to the HUI-Miner algorithm [10].

Most algorithms process the static database to mine HUIs. Ahmed et al. proposed an IHUP algorithm with three tree structures for mining HUIs with transaction insertion [6]. Lin et al. proposed an incremental algorithm (FUP-HUI-INS) [13] for updating the discovered HUIs based on the FUP concept [9] and two-phase model [16] with transaction insertion. Lin et al. then also proposed an improved pre-large concept for mining high-utility itemsets with transaction insertion (PRE-HUI-INS) [14]. Since FUP-HUI-INS and PRE-HUI-INS algorithms are processed by two-phase model, an additional database rescan is still necessary performed to find the actually HUIs. Besides, both of them required computations to find the HTWUIs based on the pattern-growth approach.

3 Preliminaries and Problem Statement

Definition 1. An itemset X is a set of k distinct items $\{i_1, i_2, \dots, i_k\}$ in a quantitative database $D = \{T_1, T_2, \dots, T_n\}$, in which k is the length of an itemset. An itemset X is contained in a transaction T_n if $X \subseteq T_n$.

Definition 2. The utility of an item i_j in T_q is defined as $u(i_j, T_q) = q(i_j, T_q) \times p(i_j)$, in which $q(i_j, T_q)$ is the quantity of an item i_j in T_q , $p(i_j)$ is the profit value of an item i_j .

Definition 3. The utility of an itemset X in transaction T_q is denoted as $u(X, T_q)$, which can be defined as:

$$u(X, T_q) = \sum_{i_j \in X \wedge X \subseteq T_q} u(i_j, T_q).$$

Definition 4. The utility of an itemset X in D is denoted as $u(X)$, which can be defined as:

$$u(X) = \sum_{X \subseteq T_q \wedge T_q \in D} u(X, T_q).$$

Definition 5. The transaction utility of transaction T_q is denoted as $tu(T_q)$, in which m is the number of items in T_q . Thus $tu(T_q)$ can be defined as:

$$tu(T_q) = \sum_{j=1}^m u(i_j, T_q).$$

Definition 6. Total utility of D is denoted as TU^D , which can be defined as:

$$TU^D = \sum_{T_q \in D} tu(T_q).$$

Definition 7. Given a database D and the minimum high utility threshold is set as ε , a high-utility itemset X in D is denoted as $HUI^D(X)$, which can be defined as:

$$HUI^D(X) = \sum_{X \subseteq T_q \wedge T_q \in D} u(X, T_q) \geq \varepsilon \times TU^D.$$

Problem Statement. Given a transactional database D , its total utility is defined as TU^D from D , a minimum utility threshold is set at $0 < \varepsilon \leq 1$, the HUIM is to find the completely k -itemsets whose utilities larger than or equal to minimum utility count as $(\varepsilon \times TU^D)$.

Since the downward-closure property of ARM is not kept in HUIM, the transaction-weighted downward closure property (TWDC) was thus proposed by two-phase model [16].

Definition 8. The transaction-weighted utility of an itemset X is the sum of all transaction utilities $tu(T_q)$ containing an itemset X , which is defined as:

$$TWU(X) = \sum_{X \subseteq T_q, T_q \in D} tu(T_q).$$

Definition 9. An itemset X is defined as a high transaction-weighted utilization itemset (HTWUI) if $TWU^D(X) \geq \varepsilon \times TU^D$.

Property 1. The transaction-weighted downward closure (TWDC) property of two-phase model is that if an itemset X is not as a HTWUI, the subsets of X could not be HUIs.

4 Proposed Incremental Algorithm

In this paper, the HUI-Miner algorithm [15] is adopted to design the incremental algorithm for HUIM. Before transactions are inserted into the original database, the utility-list structures are built in advance to keep not only the HTWUIs but also those itemsets which are not the high transaction-weighted utilization itemsets from the original database to avoid the database rescan with transaction insertion.

4.1 Utility-List Structures

Definition 10. An entry of X in the utility-list structure consisted of the set TIDs for X in T_q ($X \subseteq T_q \in D$), the set of utility for X in T_q (*Iutility*), and the set of remaining utility for X in T_q (*Rutility*), in which *Rutility* is defined as:

$$X.Rutility(T_q) = \sum_{i_j \in T_q \wedge i_j \notin X} u(i_j, T_q).$$

The construction algorithm can be found from HUI-Miner approach [15]. In the construction process, the itemsets are sorted in ascending order of their transaction-weighted utility (TWU). For the *Rutility* of an itemset X in a transaction, it keeps the rest utilities in the transaction except the processed itemset X . Since the TWU values of the itemsets are changed with transaction insertion, the sorted order of the utility-list structures and the *Rutility* value should also be changed. The number of inserted transactions is, however, very small compared to the original database. In the proposed algorithm, the sorted order of the itemsets in the inserted transactions follows the initially order of itemsets in the original database.

Definition 11. The $X.Utility.sum$ is to sum the utilities of an itemset X in database D as:

$$X.Utility.sum = \sum_{X \subseteq T_q \wedge T_q \in D} X.Utility(T_q).$$

Definition 12. The $X.Rutility.sum$ is to sum the rest utilities except an itemset X in database D as:

$$X.Rutility.sum = \sum_{X \subseteq T_q \wedge T_q \in D} X.Rutility(T_q).$$

For more k -itemsets, the utility-list structures are recursively constructed until no candidates are generated for determination.

Definition 13. Any extension of an itemset X is a combination of X with the itemset(s) after an itemset X , which is denoted as X' .

For example, assume the itemset B , then the extension of B is BC , BD and so on, both BC and BD can denoted as B' .

Based on the HUI-Miner [15], a pruning strategy can also be adopted to compress the border for determination than the TWDC property.

Property 2. Given the utility-list structure of an itemset X , if the summation of $Utility$ and $Rutility$ of an itemset X in D is less than the minimum utility count, any extension X' of X is not a HUI.

In addition, the Estimated Utility Co-occurrence Pruning (EUCP) strategy [10] is also adopted in the proposed algorithm to further keep the relationship of 2-itemsets, thus eliminating the extension itemsets with lower utility without re-constructing the utility-list structures.

4.2 Proposed Algorithm

Based on the above properties inheriting from HUI-Miner and EUCS structures, the proposed incremental algorithm is described in Fig. 1.

Algorithm 1: HUI-list-INS algorithm	
INPUT: D , the original database; d , the incremental database; TU^D , the total utility in D ; TU^d , the total utility in d ; $ptable$, the profit table; ε , the minimum utility threshold; $EUCS$, the Estimated Utility Co-Occurrence Structure; $DB.UL$, the utility list of D ; $db.UL$, the utility list of d ; $U.UL$, the utility list of U ; $X.UL$, the utility list of itemset X .	
OUTPUT: High-utility itemsets.	
BEGIN Procedure	
1. $DB.UL = NULL, db.UL = NULL, U.UL = NULL, X.UL = NULL;$	
2. FOR each T_q in D DO	
3. FOR each X in T_q DO	
4. $X.UL \leftarrow \{T_q, Utility, Rutility\}.$	
5. $EUCS \leftarrow \{X, X'\}.$	
6. END FOR	
7. END FOR	
8. $DB.UL \leftarrow \cup X.UL.$	

Fig. 1. Pseudo code of the proposed HUI-list-INS algorithm

```

9.  FOR each  $T_q$  in  $d$  DO
10.  FOR each  $X$  in  $T_q$  DO
11.     $X.UL \leftarrow \{T_q, Iutility, Rutility\}$ .
12.    update the  $TWU(X)$  in  $EUCS$ .
13.  END FOR
14. END FOR
15.  $db.UL \leftarrow \cup X.UL$ .
16. call merge-list( $DB.UL, db.UL, U.UL$ ).
17. FOR each  $X$  in  $U.UL$  DO
18.  IF  $X.Iutility.sum \geq (TU^D + TU^d) \times \epsilon$ 
19.     $HUIs \leftarrow X$ .
20.  END IF
21.  IF  $X.Iutility.sum + X.Rutility.sum \geq (TU^D + TU^d) \times \epsilon$  THEN
22.     $extULs \leftarrow null$ . //  $extULs$ , the set of utility list of all  $X$ 's 1-extensions;
23.    FOR each  $Y$  after  $X$  in  $U.UL$  DO
24.      IF  $\exists (X, Y, Z) \in EUCS$  and  $Z \geq (TU^D + TU^d) \times \epsilon$  THEN
25.         $extULs \leftarrow extULs + \text{Construct}(X.UL, Y, Z)$ .
26.      END IF
27.    END FOR
28.    call HUI-list-INS( $X, extULs, \epsilon$ ).
29.  END IF
30. END FOR
END Procedure

```

Fig. 1. (continued)

For the designed incremental algorithm with transaction insertion, the original database is firstly scanned to construct the utility-list structures for all 1-itemsets and the EUCS structure for each item (Line 2-8). Similarly, the inserted transactions are processed in the same way. Each related TWU values of items in the built EUCS are also updated by the inserted transactions (Line 9-15). The designed merge-list algorithm is used to combine the utility-list structures from the original database and inserted transactions into the updated utility-list structures (Line 16). After that, the 1-extensions of an itemset X is recursively processed (Line 17-28) by using a depth-first procedure. Each itemset X is then determined by the designed condition to check whether it is a HUI (Line 18-20). The extensions of the processed itemset are then determined by the designed condition (Line 21) for depth-first search. The updated EUCS structure is also used to prune the unpromising itemset, thus reducing the search space for mining high-utility itemsets (Line 24-26). The construction of utility-list structure algorithm is then performed to construct the $extULs$ of X . The proposed HUI-list-INS algorithm is then recursively performed to mine HUIs (Line 21-29). The algorithm is then terminated until no itemsets are generated. The construction procedure of utility-list structures are recursively processed for k -itemsets if it is necessary to process the depth-first search in the search space. The construction algorithm is similar as the algorithm shown in [15]. The proposed merge-list algorithm to combine original database and the incremental one is described in Fig. 2.

Algorithm 2: merge-list algorithm	
INPUT: $DB.UL$ is the utility-list of D ; $db.UL$ is the utility-list of d ; $U.UL$ is the utility-list of U .	
OUTPUT: $U.UL$.	
BEGIN Procedure	
1.	$U.UL = null, X.UL = null$.
2.	FOR each itemset X and $X.UL \in DB.UL$ DO
3.	IF $X.UL \neq null$ THEN
4.	search itemset $X \in DB.UL$ in $db.UL$
5.	IF $\exists (X \in DB.UL \text{ and } X \in db.UL)$ THEN
	/* E_i is the element of $X.UL$ */
6.	FOR each element $E_i \in X.UL$ and $X.UL \in db.UL$ DO
7.	$X.Iutility.sum \leftarrow X.Iutility.sum + E_i.Iutility$;
8.	$X.Rutility.sum \leftarrow X.Rutility.sum + E_i.Rutility$;
9.	$X.UL \leftarrow E_i$.
10.	END FOR
11.	END IF
12.	$U.UL \leftarrow X.UL$.
13.	END IF
14.	END FOR
15.	RETURN $U.UL$.
END Procedure	

Fig. 2. Pseudo code of the proposed merge-list algorithm

5 Experimental Evaluation

Several experiments in terms of execution time, and memory consumption are conducted to show the performance of the proposed algorithm in four database including three real-life [1] and a synthetic databases [5]. The two-phase algorithm [16], the state-of-the-art FHM algorithm [10], two incremental FUP-HUI-INS [13] and PRE-HUI-INS [14] algorithms are used to compare the proposed algorithm. The values of quantities and profits were assigned to the purchased items in all databases by the two-phase model [16] except foodmart database. Parameters and characteristics for four databases are respectively described in Tables 1 and 2.

Table 1. Parameters descriptions

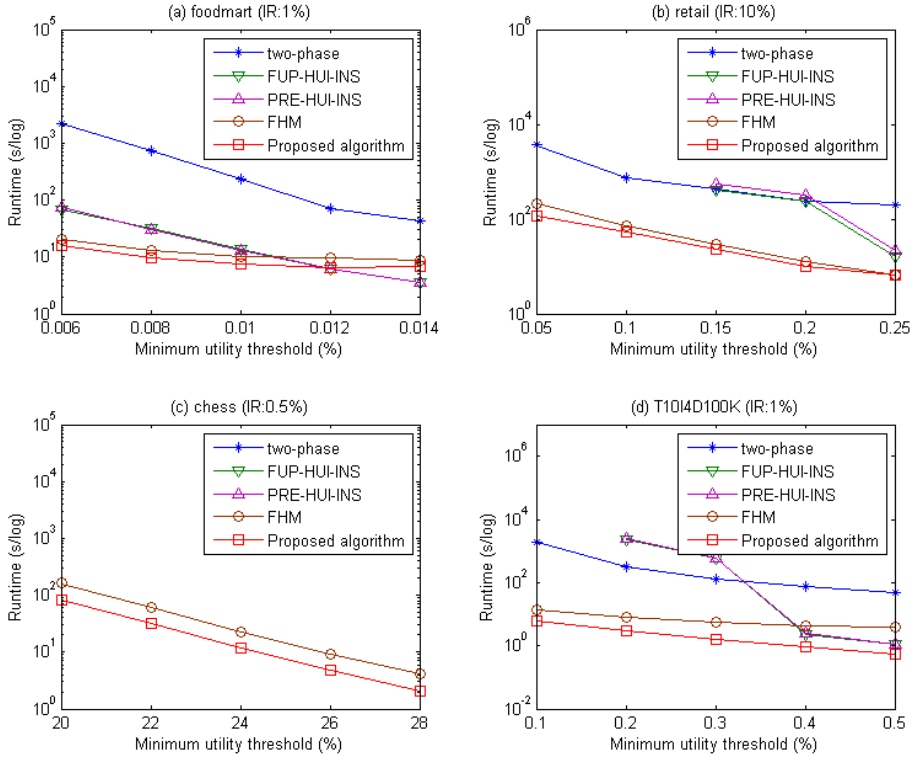
#ID	Total number of transactions
#II	Number of distinct items
AvgLen	Average length of transactions
MaxLen	Maximal length of transactions

Table 2. Characteristics of used databases

Databases	#DI	#II	AvgLen	MaxLen
foodmart	21,556	1,559	4	11
retail	88,162	16,470	10.3	76
chess	3,196	75	37	37
T10I4D100K	100,000	870	10.1	29

5.1 Runtime

Experiments were made to show the runtime of the proposed algorithm compared to the two-phase and FHM algorithms in batch mode and the other two incremental algorithms. The runtime includes the construction and mining phases. Experiments are then conducted to show the comparisons under various minimum utility thresholds (MUs) with a fixed insertion ratio (IR). The results are shown in Fig. 3.

**Fig. 3.** Runtime under various minimum utility thresholds

From Fig. 3, it can be observed that the proposed algorithm has better performance than the two-phase and FHM algorithms in batch mode and the incremental FUP-HUI-INS and PRE-HUI-IN algorithms. The runtime is decreasing along with the

increasing of MU. The observation is reasonable since less candidates of HUIs are generated when MU is set higher. When MU is set lower, the gap between the proposed algorithm and other three algorithms becomes large except the FHM algorithm, which indicates that the other three algorithms required more runtime than the proposed algorithm. Since the FHM algorithm uses the similar pruning strategies as the proposed approach, there is no great difference between them. The FHM is, however, performed in batch mode, thus requiring database rescan each time when the transactions are inserted into the original database. Experiments are then conducted to show the comparisons under different IRs with a fixed MU. The results are shown in Fig. 4.

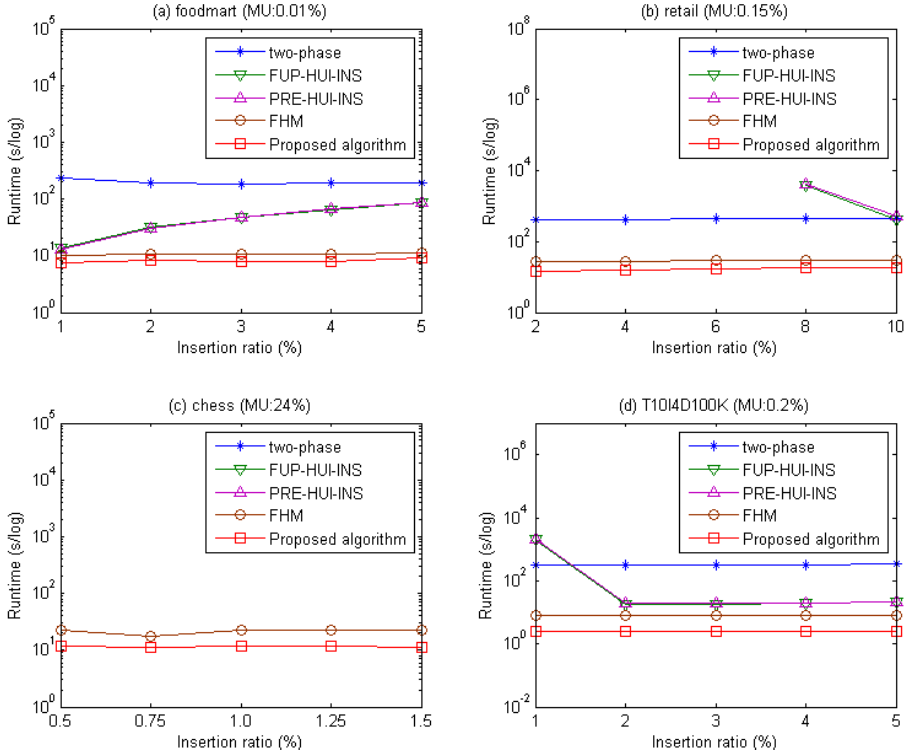


Fig. 4. Runtime under various insertion ratios

From Fig. 4, it also can be observed that the proposed algorithm outperforms the other algorithms under various IRs. Take an example of Fig. 4(b), the MU is set at 0.15%, and the IRs are respectively set from 2% to 10%, with 2% increments each time. Two incremental FUP-HUI-INS and PRE-HUI-INS algorithms have worse performance than the other algorithms. When the IR is set lower than 8%, the average runtime of two-phase algorithm is 420 seconds, the FHM is 28 seconds, and the proposed algorithm is 16 seconds. The runtime of FUP-HUI-INS and PRE-HUI-INS algorithms exceeds 10^4 seconds. The reason is that FUP-HUI-INS and PRE-HUI-INS algorithms

could have “combination explosion” problem when MU or IR is set lower. This situation may frequently occur depending on the database characteristics. From the above results, the other algorithms have worse performance in chess database except the FHM and the proposed algorithm, which can be easily observed from Fig. 3(c) and Fig. 4(c). Since the chess belongs to dense database with long patterns in the transactions, a great amount of HTWUIs are generated by the two-phase, FUP-HUI-INS and PRE-HUI-INS algorithms.

5.2 Memory Consumption

Memory consumption of the propose algorithm compared to the other algorithms is then evaluated. Experiments are then conducted to show the comparisons under various MUs with a fixed IR. The results are shown in Fig. 5.

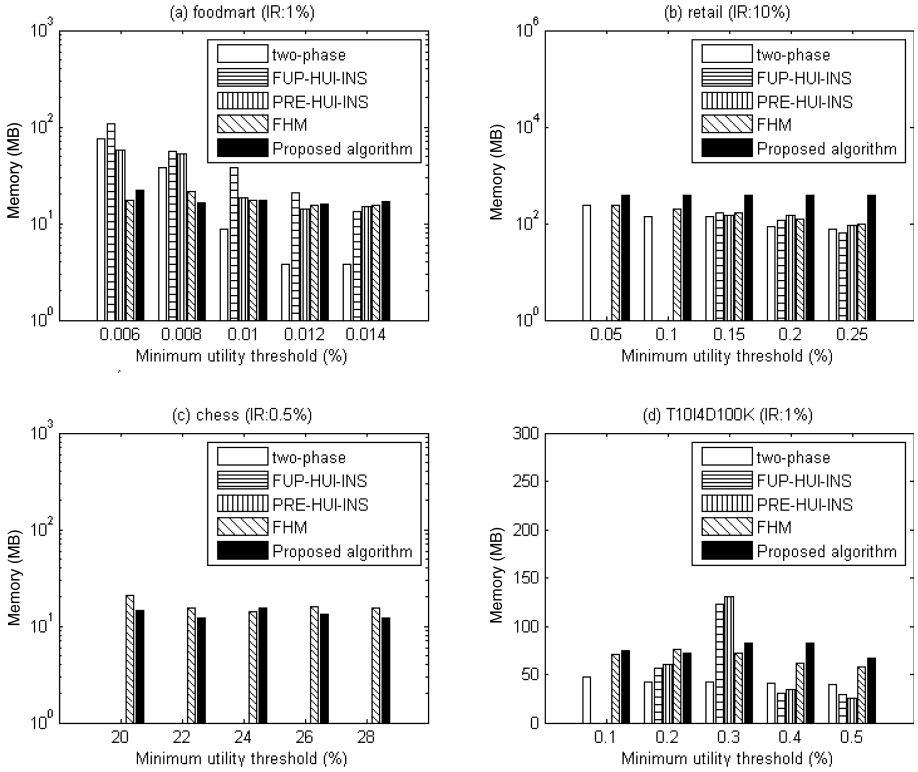


Fig. 5. Memory consumption under various minimum utility thresholds

From Fig. 5, it can be observed that the FHM and the proposed algorithms requires steady memory along with the increasing of MUs compared to the other algorithms. This is because that the FHM and the proposed algorithms are necessary to build the utility-list structures for keep the itemsets. When MU is set lower, the proposed algorithm requires fewer memory than the other algorithms, which can be observed from Fig. 5(a). Experiments are then conducted to show the comparisons under various IRs with a fixed MU. The results are shown in Fig. 6.

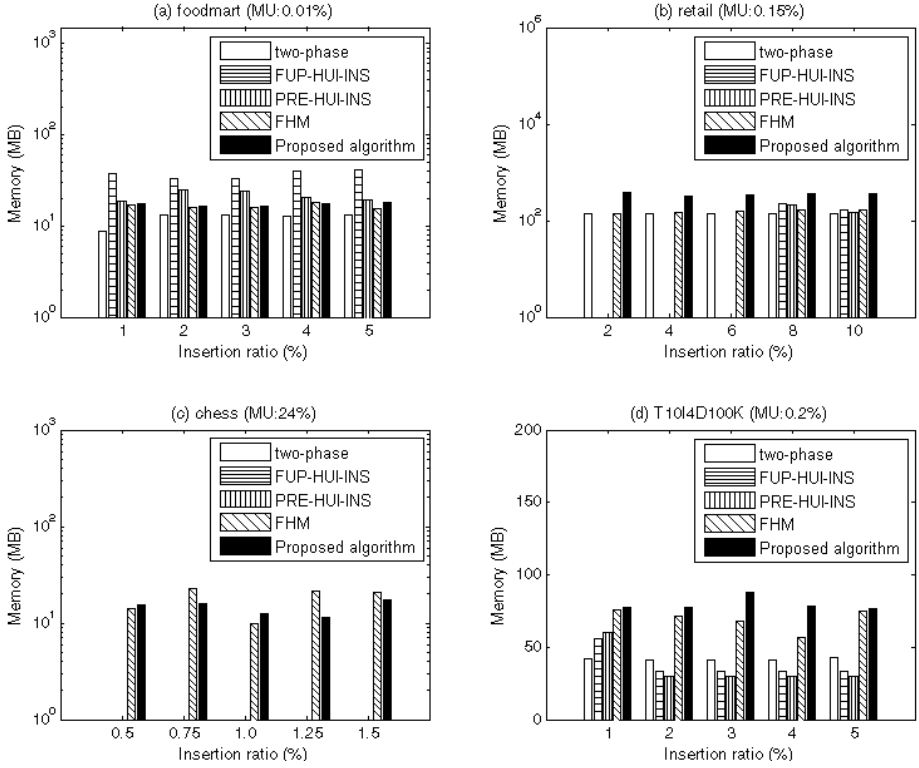


Fig. 6. Memory consumption under various insertion ratios

From Fig. 6(a), it can be observed that the proposed algorithm requires less memory than the other incremental algorithms along with the increasing of IRs. From Fig. 6(b) and (d), it can be observed that the proposed algorithm requires more memory than the other algorithms. This is reasonable since more itemsets are kept in the proposed algorithm for later incremental database.

6 Conclusion

In this paper, a novel incremental algorithm is proposed to maintain and update the built utility-list structures for mining HUIs with transaction insertion. Based on the utility-list structures, related information in the original database can thus be compressed. The proposed algorithm also applies the Estimated Utility Co-Occurrence Structure (EUCS) to keep the information between 2-itemsets, thus speeding up the computations. Without the level-wise approach for generate-and-test candidates, HUIs can be easily discovered based on the designed algorithm for the incremental database. Experimental results show that the performance of the proposed algorithm outperforms than other algorithms.

Acknowledgement. This research was partially supported by the Shenzhen Peacock Project, China, under grant KQC201109020055A, by the Natural Scientific Research Innovation Foundation in Harbin Institute of Technology under grant HIT.NSRIF.2014100, and by the Shenzhen Strategic Emerging Industries Program under grant ZDSY20120613125016389.

References

1. Frequent itemset mining dataset repository (2012), <http://fimi.ua.ac.be/data/>
2. Abdullah, Z., Herawan, T., Deris, M.: Mining Significant Least Association Rules using Fast SLP-growth Algorithm. In: Kim, T.-h., Adeli, H. (eds.) AST/UCMA/ISA/ACN 2010. LNCS, vol. 6059, pp. 324–336. Springer, Heidelberg (2010)
3. Agrawal, R., Imielinski, T., Swami, A.: Database Mining: A Performance Perspective. *IEEE Transactions on Knowledge and Data Engineering* 5, 914–925 (1993)
4. Agrawal, R., Srikant, R.: Fast Algorithms for Mining Association Rules in Large Databases. In: The International Conference on Very Large Data Bases, pp. 487–499 (1994)
5. Agrawal, R., Srikant, R.: Quest synthetic data generator (1994), <http://www.Almaden.ibm.com/cs/quest/syndata.html>
6. Ahmed, C.F., Tanbeer, S.K., Jeong, B.S., Lee, Y.K.: Efficient Tree Structures for High Utility Pattern Mining in Incremental Databases. *IEEE Transactions on Knowledge and Data Engineering* 21, 1708–1721 (2009)
7. Chan, R., Yang, Q., Shen, Y.D.: Mining high utility itemsets. In: IEEE International Conference on Data Mining, pp. 19–26 (2003)
8. Chen, M.S., Han, J., Yu, P.S.: Data Mining: An Overview from a Database Perspective. *IEEE Transactions on Knowledge and Data Engineering* 8, 866–883 (1996)
9. Cheung, D.W., Han, J., Ng, V., Wong, C.Y.: Maintenance of Discovered Association Rules in Large Databases: An Incremental Updating Technique. In: International Conference on Data Engineering, pp. 106–114 (1996)
10. Fournier-Viger, P., Wu, C.W., Zida, S., Tseng, V.S.: FHM: Faster High-Utility Itemset Mining Using Estimated Utility Co-Occurrence Pruning. *Foundations of Intelligent Systems* 8502, 83–92 (2014)
11. Hong, T.P., Lin, C.W., Wu, Y.L.: Incrementally Fast Updated Frequent Pattern Trees. *Expert Systems with Applications* 34, 2424–2435 (2008)
12. Lin, C.W., Hong, T.P., Lu, W.H.: An Effective Tree Structure for Mining High Utility Itemsets. *Expert Systems with Applications* 38, 7419–7424 (2011)
13. Lin, C.W., Lan, G.C., Hong, T.P.: An Incremental Mining Algorithm for High Utility Itemsets. *Expert Systems with Applications* 39, 7173–7180 (2012)
14. Lin, C.W., Hong, T.P., Lan, G.C., Wong, J.W., Lin, W.Y.: Incrementally Mining High Utility Patterns based on Pre-large Concept. *Applied Intelligence* 40, 343–357 (2014)
15. Liu, M., Qu, J.: Mining High Utility Itemsets without Candidate Generation. In: ACM International Conference on Information and Knowledge Management, pp. 55–64 (2012)
16. Liu, Y., Liao, W.K., Choudhary, A.: A Two-Phase Algorithm for Fast Discovery of High Utility Itemsets. In: Ho, T.-B., Cheung, D., Liu, H. (eds.) PAKDD 2005. LNCS (LNAI), vol. 3518, pp. 689–695. Springer, Heidelberg (2005)
17. Nath, B., Bhattacharyya, D.K., Ghosh, A.: Incremental Association Rule Mining: A Survey. *WIREs Data Mining Knowledge Discovery* 3, 157–169 (2013)

18. Tseng, V.S., Shie, B.E., Wu, C.W., Yu, P.S.: Efficient Algorithms for Mining High Utility Itemsets from Transactional Databases. *IEEE Transactions on Knowledge and Data Engineering* 25, 1772–1786 (2013)
19. Wu, C.W., Shie, B.E., Yu, P.S., Tseng, V.S.: Mining Top-k High Utility Itemsets. In: *ACM International Conference on Knowledge Discovery and Data Mining*, pp. 12–16 (2012)
20. Yao, H., Hamilton, H.J.: Mining Itemset Utilities from Transaction Databases. *Data & Knowledge Engineering* 59, 603–626 (2006)